

Lab Manual

Programming for AI

AI - 212



By

Wishaq Akbar

2023-BSAI-041

Submitted to

Mr. Sumraiz Zahid

Bachelor of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCES

Content

[illegible]

Lab 01 : Python Basics

Question 1:

This program simulates an ATM system that validates PIN attempts, processes withdrawals with service charges, and blocks the card after multiple invalid attempts.

Code:

```
c balance = 5000
pin = "1234"
attempts = 0

while attempts < 3:
    entered = input("Enter PIN: ")
    if entered == pin:
        amt = int(input("Enter withdrawal amount: "))
        charge = amt * 0.02
        total = amt + charge
        if total <= balance:
            balance -= total
            print("Remaining Balance:", balance)
        else:
            print("Insufficient Balance")
            break
    else:
        attempts += 1
        print("Invalid PIN")
else:
    print("Card Blocked")
```

Remaining Balance: 3920.0

Question 2:

This program manages grocery inventory, applies discounts, calculates tax, and prints the final bill.

Code:

```
store = {  
    "Rice": {"price": 200, "stock": 10},  
    "Sugar": {"price": 150, "stock": 8}  
}  
  
total = 0  
item = "Rice"  
qty = 2  
  
if store[item]["stock"] >= qty:  
    cost = store[item]["price"] * qty  
    store[item]["stock"] -= qty  
    total += cost  
  
discount = 0.1  
total -= total * discount  
total += total * 0.05  
print(total)
```



378.0

Question 3:

This program calculates weighted average marks, assigns grades, and checks scholarship eligibility.

Code:

```
marks = [80, 75, 90, 85, 70]  
weights = (0.2, 0.2, 0.2, 0.2, 0.2)  
  
weighted = sum(m*w for m, w in zip(marks, weights))  
  
if weighted >= 80:  
    grade = "A"  
elif weighted >= 70:
```

```
        grade = "B"
    elif weighted >= 60:
        grade = "C"
    else:
        grade = "Fail"

    print(weighted, grade, weighted >= 80)
```

```
80.0 A True
```

Question 4:

This program calculates BMI using weight and height and provides health advice based on BMI and age.

Code:

```
weight = 80
height = 1.7
age = 30

bmi = weight / (height ** 2)

if bmi >= 30:
    advice = "Diet Plan Required"
elif bmi < 18.5:
    advice = "Exercise Recommended"
else:
    advice = "Healthy"

print(bmi, advice)
```

```
27.68 Healthy
```

Question 5:

This program deducts telecom usage charges and reminds users when balance is low.

Code:

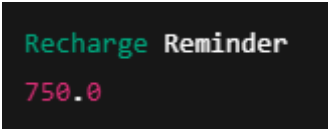
```
rates = {"call": 2, "sms": 1, "data": 0.5}
balance = 1000
```

```
usage = {"call": 50, "sms": 20, "data": 100}

for k in usage:
    balance -= usage[k] * rates[k]

if balance < 200:
    print("Recharge Reminder")

print(balance)
```



```
Recharge Reminder
750.0
```

Question 6:

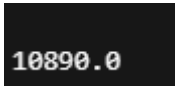
This program applies membership-based and amount-based discounts and calculates tax.

Code:

```
bill = 12000
member = "Gold"

discount = 0.1 if member == "Gold" else 0
if bill > 10000:
    discount += 0.05

bill -= bill * discount
bill += bill * 0.05
print(bill)
```



```
10890.0
```

Question 7:

This program classifies patients based on vital signs using nested conditions.

Code:

```
temp = 39
oxygen = 85
```

```
if oxygen < 90:
    status = "Critical"
elif temp > 38:
    status = "Urgent"
else:
    status = "Stable"

print(status)
```

Critical

Question 8:

This program calculates exam percentage with negative marking and assigns results.

Code:

```
total = 100
obtained = 70
wrong = 10

score = obtained - (wrong * 0.25)
percent = (score / total) * 100

if percent >= 50:
    result = "Pass"
else:
    result = "Fail"

print(percent, result)
```

67.5 Pass

Question 9:

This program calculates ticket price based on age, timing, and ticket type.

Code:

```
price = 1000
age = 65
peak = True
```

```
if age >= 60:
    price -= price * 0.3
if peak:
    price += price * 0.1

print(price)
```

```
770.0
```

Question 10:

This program suggests outfits based on weather, temperature, and event type.

Code:

```
temp = 38
weather = "Sunny"
event = "Business"

if event == "Business" and temp > 35:
    print("Light formal suit")
```

```
Light formal suit
```


Lab 02 : OOP Basics

Question 1:

This program demonstrates a BankAccount class that stores account details and allows depositing, withdrawing, and displaying balance while preventing overdrafts.

Code:

```
class BankAccount:
    def __init__(self, acc_no, name, balance):
        self.acc_no = acc_no
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient balance")

    def display_balance(self):
        print("Balance:", self.balance)

acc = BankAccount(101, "Wishaq", 5000)
acc.deposit(2000)
acc.withdraw(3000)
acc.display_balance()
```

Output:

Balance: 4000

Question 2:

This program models a library system using Book and Library classes to manage books, search records, and track availability.

Code:

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def search(self, keyword):
        for b in self.books:
            if keyword.lower() in b.title.lower() or keyword.lower()
in b.author.lower():
                print(b.title, "-", "Available" if b.available else
"Not Available")

    def checkout(self, isbn):
        for b in self.books:
            if b.isbn == isbn and b.available:
                b.available = False

    def return_book(self, isbn):
        for b in self.books:
            if b.isbn == isbn:
                b.available = True

lib = Library()
```

```
b1 = Book("Python Basics", "John", "111")
lib.add_book(b1)
lib.search("Python")
lib.checkout("111")
lib.search("Python")
```

Output:

```
Python Basics - Available
Python Basics - Not Available
```

Question 3:

This program models a library system using Book and Library classes to manage books, search records, and track availability.

Code:

```
class Student:
    def __init__(self, sid, name):
        self.sid = sid
        self.name = name
        self.grades = []

    def add_grade(self, g):
        self.grades.append(g)

    def average(self):
        return sum(self.grades) / len(self.grades)

    def letter_grade(self):
        avg = self.average()
        if avg >= 80:
            return "A"
        elif avg >= 70:
            return "B"
        elif avg >= 60:
            return "C"
        else:
```

```
        return "F"

    def display(self):
        print(self.name, self.average(), self.letter_grade())

s = Student(1, "Wishaq")
s.add_grade(85)
s.add_grade(90)
s.display()
```

Output:

```
Wishaq 87.5 A
```

Question 4:

This program simulates a food ordering system where items are added to an order and total cost is calculated with tax.

Code:

```
class MenuItem:
    def __init__(self, name, price):
        self.name = name
        self.price = price

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def total(self):
        subtotal = sum(i.price for i in self.items)
        tax = subtotal * 0.1
        return subtotal + tax
```

```
order = Order()
order.add_item(MenuItem("Burger", 500))
order.add_item(MenuItem("Drink", 150))
print(order.total())
```

Output:

```
715.0
```

Question 5:

This program demonstrates vehicle rental functionality using a base Vehicle class to calculate rental cost.

Code:

```
class Vehicle:
    def __init__(self, plate, price):
        self.plate = plate
        self.price = price
        self.available = True

    def rent(self, days):
        if self.available:
            self.available = False
            return self.price * days
        return 0

car = Vehicle("ABC123", 3000)
print(car.rent(3))
```

Output:

```
9000
```

Question 6:

This program models an employee payroll system to calculate monthly and annual salaries.

Code:

```
class Employee:
    def __init__(self, eid, name, salary):
        self.eid = eid
        self.name = name
```

```
        self.salary = salary

    def monthly(self):
        return self.salary

    def annual(self):
        return self.salary * 12

e = Employee(1, "Ahmed", 50000)
print(e.monthly())
print(e.annual())
```

Output:

```
50000
600000
```

Question 6:

This program implements a shopping cart system that manages products, calculates total cost, and updates stock.

Code:

```
class Product:
    def __init__(self, pid, name, price, stock):
        self.pid = pid
        self.name = name
        self.price = price
        self.stock = stock

class ShoppingCart:
    def __init__(self):
        self.cart = []

    def add(self, product):
        if product.stock > 0:
            self.cart.append(product)
            product.stock -= 1
```

```
def total(self):  
    return sum(p.price for p in self.cart)  
  
p = Product(1, "Laptop", 80000, 2)  
cart = ShoppingCart()  
cart.add(p)  
print(cart.total())
```

Output:

```
80000
```

Question 8:

This program manages patient records and doctor appointments with scheduling and viewing functionality.

Code:

```
class Patient:  
    def __init__(self, pid, name):  
        self.pid = pid  
        self.name = name  
  
class Appointment:  
    def __init__(self):  
        self.appointments = []  
  
    def schedule(self, patient, date):  
        self.appointments.append((patient.name, date))  
  
    def view(self):  
        print(self.appointments)  
  
p = Patient(1, "Wishaq")  
a = Appointment()  
a.schedule(p, "10-01-2025")  
a.view()
```

Output:

```
[(Wishaq, '10-01-2025')]
```

Question 9:

This program simulates a social media user who can create posts and view their timeline.

Code:

```
class User:
    def __init__(self, username):
        self.username = username
        self.posts = []
    def post(self, text):
        self.posts.append(text)

    def timeline(self):
        print(self.posts)
u = User("wishaq")
u.post("Hello World")
u.timeline()
```

Output:

```
['Hello World']
```

Question 10:

This program simulates a social media user who can create posts and view their timeline.

Code:

```
class Room:
    def __init__(self, price):
        self.price = price
class Booking:
    def calculate(self, room, days):
        return room.price * days
r = Room(7000)
b = Booking()
print(b.calculate(r, 3))
```

Output:

```
21000
```


Lab 03 : Numpy

Question 1:

This program analyzes temperature data using NumPy to calculate average, minimum, maximum, and temperature range.

Code:

```
import numpy as np

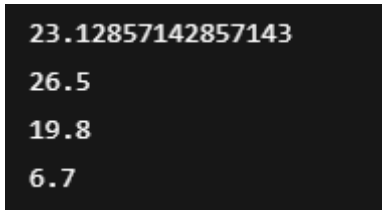
temps = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])

print(np.mean(temps))

print(np.max(temps))

print(np.min(temps))

print(np.max(temps) - np.min(temps))
```



```
23.12857142857143
26.5
19.8
6.7
```

Question 2:

This program stores weekly apple sales data and determines totals, peak sale day, and best-performing week.

Code:

```
import numpy as np

sales = np.array([
    [12,15,14,10,9,8,7],
    [11,12,13,9,5,8,6],
    [7,9,12,10,11,13,12],
    [10,11,12,7,8,9,11]
])

weekly_totals = np.sum(sales, axis=1)

print(weekly_totals)

print(np.unravel_index(np.argmax(sales), sales.shape))

print(np.argmax(weekly_totals))
```

Question 3:

This program adds bonus marks using broadcasting and computes statistics such as median and standard deviation.

Code:

```

import numpy as np
marks = np.array([56,78,45,90,88,76,59,62])
marks += 5
print(marks)
print(np.sum(marks > 60))
print(np.median(marks))
print(np.std(marks))

```

```

[61 83 50 95 93 81 64 67]
6
74.0
14.53

```

Question 4:

This program analyzes daily step data to compute weekly averages and activity statistics.

Code:

```

import numpy as np

np.random.seed(0)
steps = np.random.randint(2000,15000,30)
steps = steps.reshape(5,6)

print(np.mean(steps, axis=1))
print(np.unravel_index(np.argmax(steps), steps.shape))
print(np.sum(steps > 10000))

```

```

[ 9424.5  8407.5  8811.  10805.5 10244.5]
(3, 4)
14

```

Question 5:

This program analyzes heart rate readings to detect abnormal values and overall extremes.

Code:

```

import numpy as np

np.random.seed(1)
heart = np.random.randint(60,150,(10,4))

```

```

print(np.mean(heart, axis=1))
print(heart > 120)
print(np.max(heart))
print(np.min(heart))

```

```

[107.25  98.25 101.5  106.25 105.75 100.75 108.75 103.75 108.   97.75]
[[False  True False False] ... ]
149
60

```

Question 6:

This program compares sales performance between two branches and computes correlation.

Code:

```

import numpy as np

A = np.array([122,135,150,160,155,140])
B = np.array([110,130,148,165,158,145])

print(A - B)
print(np.mean(A))
print(np.mean(B))
print("A" if np.sum(A) > np.sum(B) else "B")
print(np.corrcoef(A,B)[0,1])

```

```

[12  5  2 -5 -3 -5]
143.67
142.67
A
0.98

```

Question 7:

This program normalizes image pixel values, applies transformations, and analyzes brightness.

Code:

```

import numpy as np

img = np.array([

```

```

        [100,120,130,90],
        [80,110,140,100],
        [70,90,120,130],
        [110,140,150,160]
    ])

    norm = img / 255
    trans = np.clip(img * 1.2, 0, 255)

    print(np.mean(img))
    print(np.mean(trans))

```

```

115.0
138.0

```

Question 8:

This program standardizes a dataset and verifies mean and variance.

Code:

```

import numpy as np

data = np.random.randint(1,101,(100,4))
mean = np.mean(data, axis=0)
std = np.std(data, axis=0)

scaled = (data - mean) / std

print(np.mean(scaled, axis=0))
print(np.var(scaled, axis=0))

```

```

[ 0. -0.  0.  0.]
[1.  1.  1.  1.]

```

Question 9:

This program analyzes traffic density data and applies traffic capping and energy calculations.

Code:

```

import numpy as np

```

```

np.random.seed(2)

traffic = np.random.randint(10,80,(24,6))

print(np.mean(traffic, axis=1))
print(np.argmax(np.sum(traffic, axis=0)))
traffic = np.clip(traffic,0,50)
print(np.sum(traffic * 0.5))

```

```

[36.83333333 51.16666667 50.16666667 66.83333333 54.33333333 44.5
 44.83333333 35.          45.5          49.33333333 45.33333333 41.
 50.          53.16666667 42.66666667 46.16666667 51.66666667 51.
 40.5         40.66666667 41.16666667 44.66666667 39.          24.66666667]
3
2836.0

```

Question 10:

This program transforms force vectors, computes magnitudes, and performs eigenvalue decomposition.

Code:

```

import numpy as np

F = np.array([
    [5,2,1],
    [3,1,4],
    [6,3,2],
    [4,2,5],
    [7,1,3]
])

T = np.array([
    [2,1,0],
    [1,3,1],
    [0,2,2]
])

```

```
F_new = F @ T  
mag = np.linalg.norm(F_new, axis=1)
```

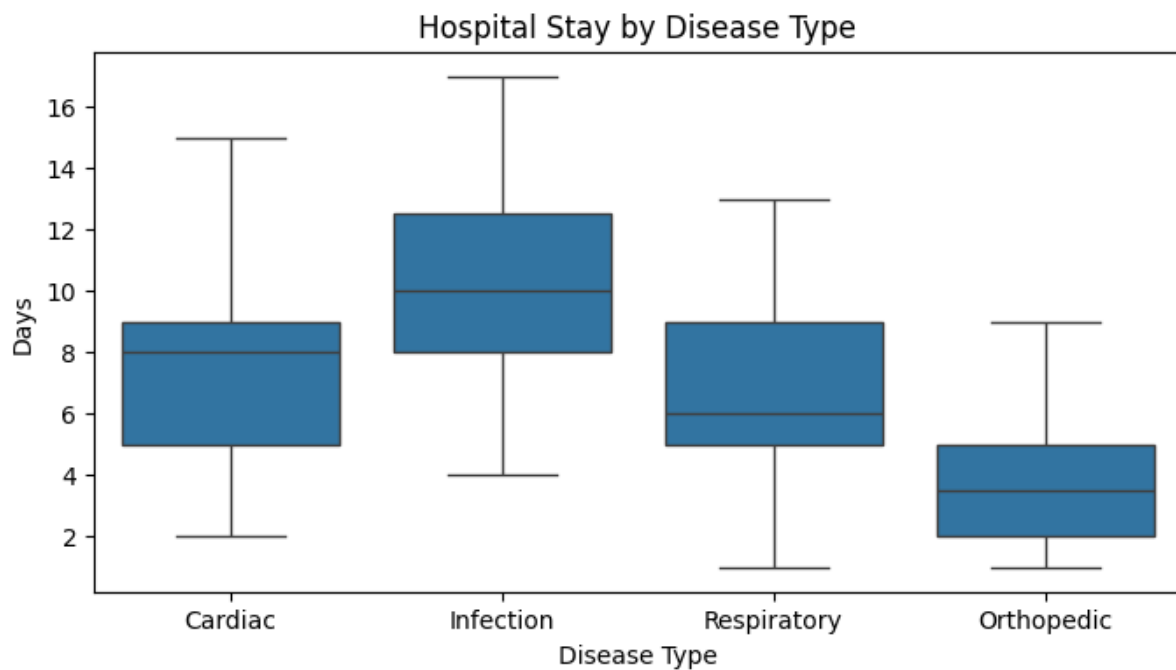
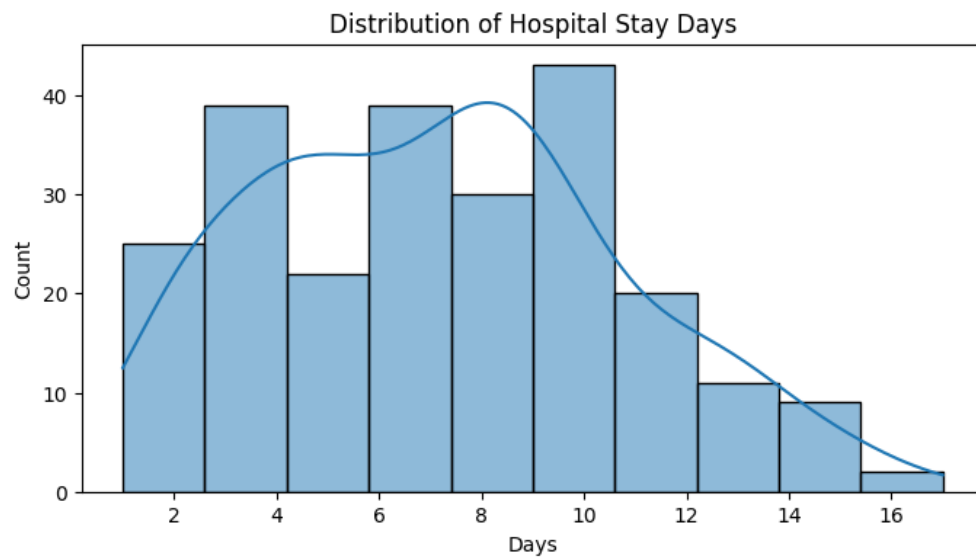
```
print(F_new)  
print(mag)  
print(np.argmax(mag))  
print(np.linalg.eig(T)[0])
```

```
[[12 13  4]  
 [ 7 12  9]  
 [15 17  7]  
 [10 16 12]  
 [15 14  7]]  
[18.38 16.19 24.1  22.36 21.47]  
2  
[4.73 1.53 0.74]
```

Lab 14 : Charts and Graph Visualization

Q1. Patient Stay Analysis (Healthcare)

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(0)
stay_map={'Cardiac':8,'Respiratory':6,'Orthopedic':4,'Infection':10}
diseases=list(stay_map.keys())
n=240
choices=np.random.choice(diseases,n)
df=pd.DataFrame({'age':np.random.randint(20,90,n),'disease_type':choices})
df['hospital_stay_days']=[np.random.poisson(lam=stay_map[d]) for d
in df['disease_type']]
plt.figure(figsize=(8,4))
sns.histplot(df['hospital_stay_days'],kde=True)
plt.title('Distribution of Hospital Stay Days')
plt.xlabel('Days')
plt.show()
plt.figure(figsize=(8,4))
sns.boxplot(x='disease_type',y='hospital_stay_days',data=df)
plt.title('Hospital Stay by Disease Type')
plt.xlabel('Disease Type')
plt.ylabel('Days')
plt.show()
med=df.groupby('disease_type')['hospital_stay_days'].median()
print(med.sort_values(ascending=False).head(1))
```



Q2. Sales Distribution Comparison (Business)

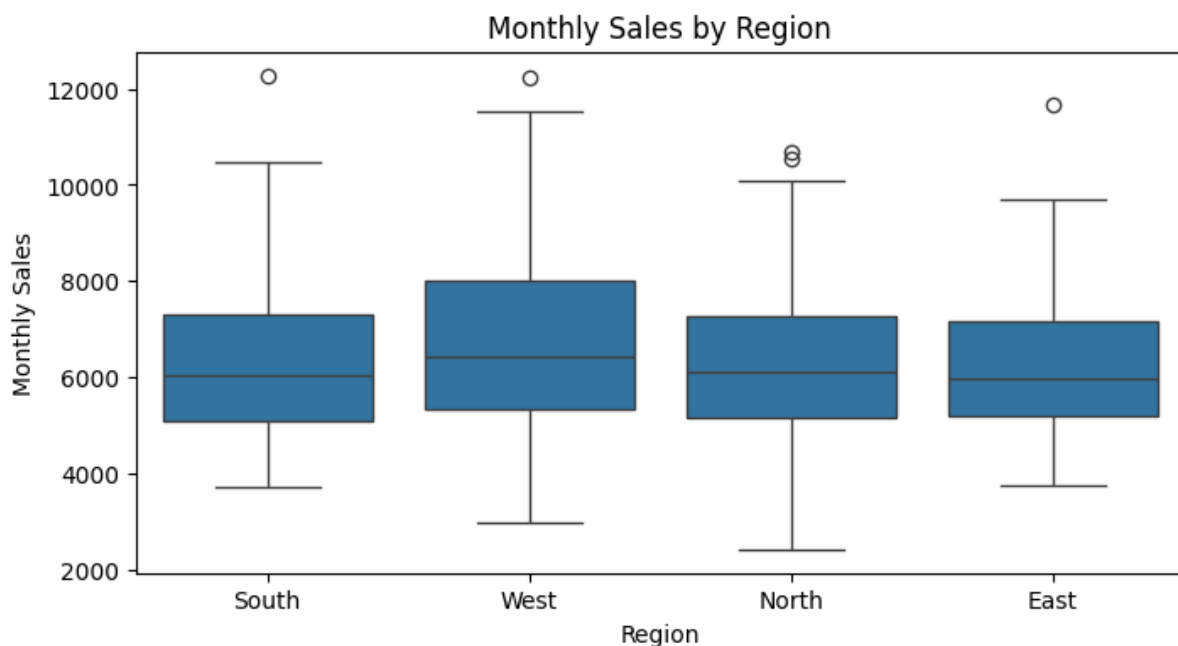
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(1)
regions=['North','South','East','West']
n=300
```

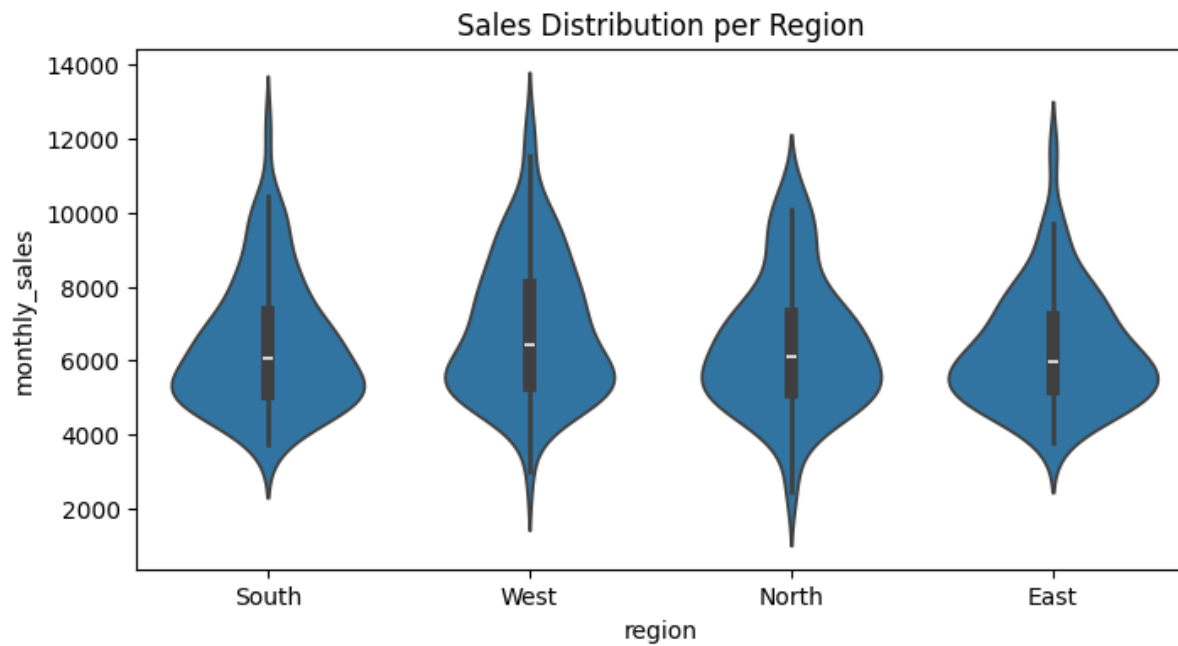


```

df=pd.DataFrame({'region':np.random.choice(regions,n),'monthly_sales':np.concatenate([
    np.random.normal(5000,400,75),
    np.random.normal(7000,1500,75),
    np.random.normal(6000,800,75),
    np.random.normal(8000,2000,75)])})
plt.figure(figsize=(8,4))
sns.boxplot(x='region',y='monthly_sales',data=df)
plt.title('Monthly Sales by Region')
plt.xlabel('Region')
plt.ylabel('Monthly Sales')
plt.show()
plt.figure(figsize=(8,4))
sns.violinplot(x='region',y='monthly_sales',data=df)
plt.title('Sales Distribution per Region')
plt.show()
print(df.groupby('region')['monthly_sales'].std().sort_values(
ascending=False))

```





West	1849.893327
North	1748.252828
South	1724.257712
East	1530.570484

Q3. Student Performance Correlation (Education)

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(2)

n=200

study_hours=np.random.normal(5,2,n).clip(0)
attendance=np.random.normal(80,10,n).clip(40,100)

final_score=(study_hours*6 + attendance*0.4 +
np.random.normal(0,8,n)).clip(0,100)

df=pd.DataFrame({'study_hours':study_hours,'attendance':attendance,'
final_score':final_score})

sns.pairplot(df)

plt.show()

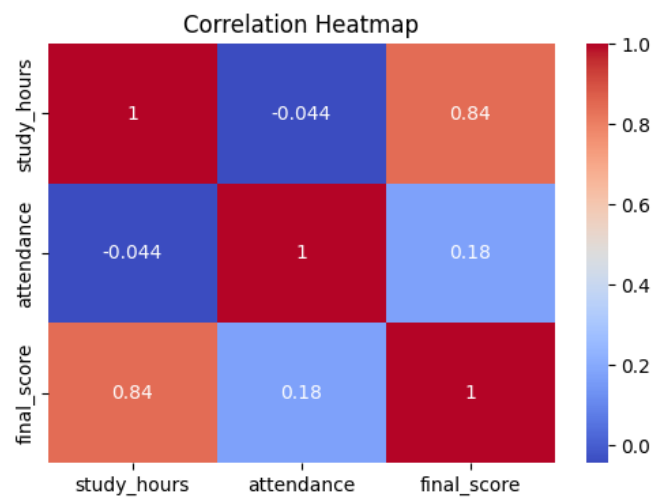
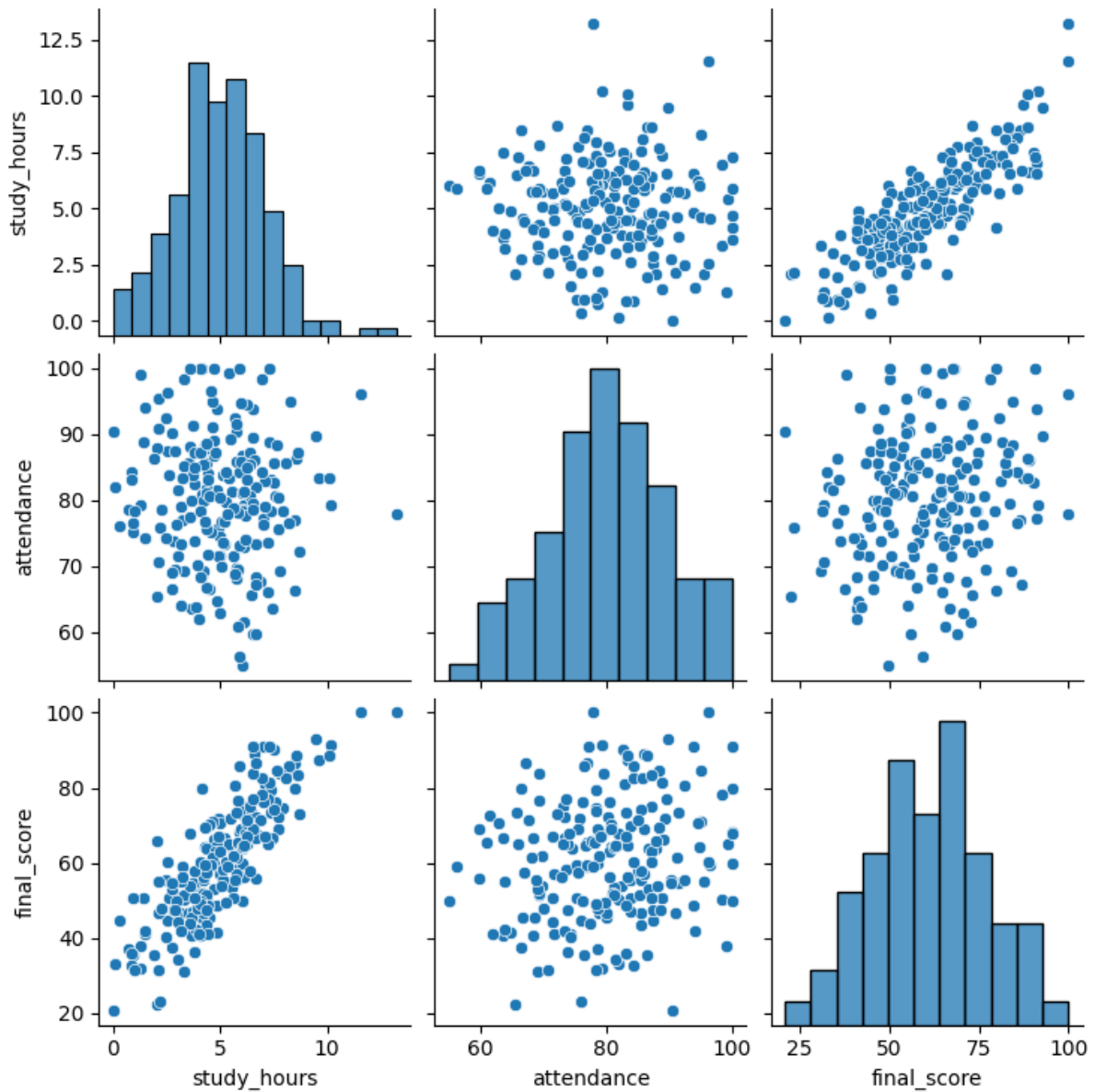
plt.figure(figsize=(6,4))

sns.heatmap(df.corr(),annot=True,cmap='coolwarm')

plt.title('Correlation Heatmap')
```

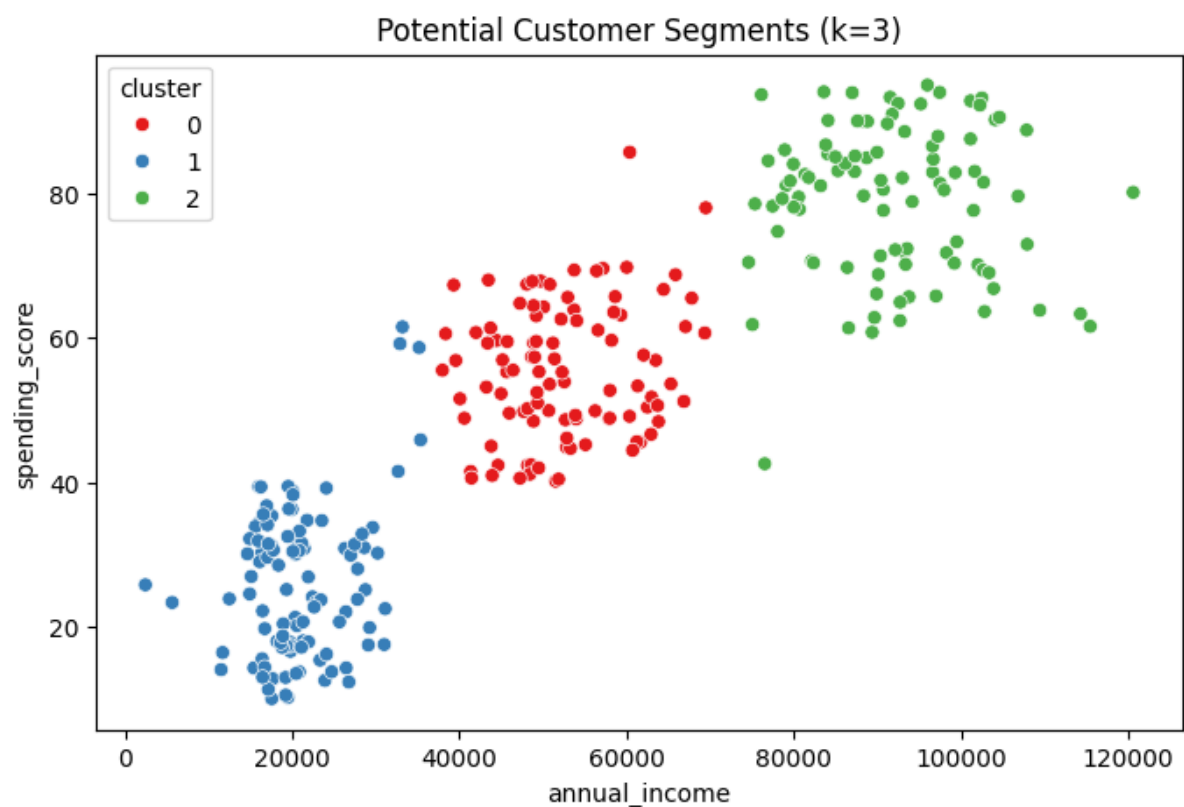
```
plt.show()
```

```
print(df.corr()['final_score'].sort_values(ascending=False))
```



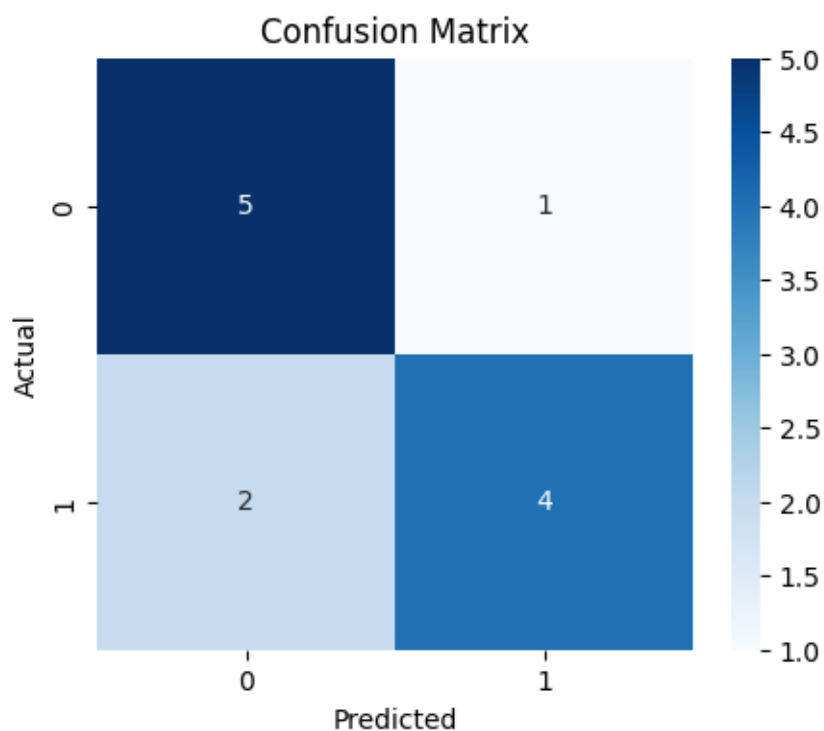
Q4. Customer Segmentation Visualization

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
np.random.seed(3)
n=300
age=np.random.randint(18,70,n)
annual_income=np.concatenate([np.random.normal(20000,5000,100),np.random.normal(50000,8000,100),np.random.normal(90000,12000,100)])
spending_score=np.concatenate([np.random.uniform(10,40,100),np.random.uniform(40,70,100),np.random.uniform(60,95,100)])
df=pd.DataFrame({'age':age,'annual_income':annual_income,'spending_score':spending_score})
plt.figure(figsize=(8,5))
sns.scatterplot(x='annual_income',y='spending_score',hue='spending_score',palette='viridis',data=df)
plt.title('Customer Spending vs Income')
plt.show()
km=KMeans(n_clusters=3,random_state=0).fit(df[['annual_income','spending_score']])
df['cluster']=km.labels_
plt.figure(figsize=(8,5))
sns.scatterplot(x='annual_income',y='spending_score',hue='cluster',palette='Set1',data=df)
plt.title('Potential Customer Segments (k=3)')
plt.show()
```



Q5. Model Result Visualization

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
y_true=np.array([0,0,1,1,0,1,0,1,1,0,1,0])
y_pred=np.array([0,1,1,1,0,0,0,1,0,0,1,0])
cm=confusion_matrix(y_true,y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues',cbar=True)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Q6. Telecom Customer Churn Prediction

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
np.random.seed(4)

n=500

contract=np.random.choice(['Month-to-month','One year','Two
year'],n,p=[0.6,0.25,0.15])

tenure=np.random.randint(0,72,n)

monthly_charges=np.random.normal(60,20,n).clip(10)

prob=np.where(contract=='Month-to-
month',0.3,0.1)+np.where(tenure<12,0.2,0)

churn=np.random.binomial(1,prob.clip(0,0.9))

df=pd.DataFrame({'tenure':tenure,'monthly_charges':monthly_charges,'
contract_type':contract,'churn':churn})

df=pd.get_dummies(df,columns=['contract_type'],drop_first=True)

X=df.drop('churn',axis=1)

y=df['churn']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,ran
dom_state=0)

model=LogisticRegression(max_iter=500)

model.fit(X_train,y_train)

y_pred=model.predict(X_test)

print('Accuracy:',accuracy_score(y_test,y_pred))

print('Confusion matrix:\n',confusion_matrix(y_test,y_pred))
```

```
Accuracy: 0.7
Confusion matrix:
[[70  1]
 [29  0]]
```

Q7. House Price Prediction

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
np.random.seed(5)
n=300
area=np.random.randint(400,4000,n)
bedrooms=np.random.randint(1,6,n)
location_score=np.random.uniform(1,10,n)
price=area*120 + bedrooms*10000 + location_score*5000 +
np.random.normal(0,20000,n)
df=pd.DataFrame({'area':area,'bedrooms':bedrooms,'location_score':lo
cation_score,'price':price})
X=df[['area','bedrooms','location_score']]
y=df['price']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,ran
dom_state=0)
model=LinearRegression()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print('MSE:',mean_squared_error(y_test,y_pred))
```

MSE: 360506875.4681977

Q8. Disease Classification System

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score,recall_score,f1_score
```



```

np.random.seed(6)

n=400

glucose=np.random.normal(110,30,n).clip(50,300)
bmi=np.random.normal(28,6,n).clip(15,50)
age=np.random.randint(18,80,n)
outcome=(glucose>125).astype(int)

df=pd.DataFrame({'glucose':glucose,'BMI':bmi,'age':age,'outcome':outcome})

scaler=StandardScaler()

X=scaler.fit_transform(df[['glucose','BMI','age']])
y=df['outcome']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)

model=DecisionTreeClassifier(random_state=0)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

print('Precision:',precision_score(y_test,y_pred))
print('Recall:',recall_score(y_test,y_pred))
print('F1:',f1_score(y_test,y_pred))

```

```

Precision: 1.0
Recall: 1.0
F1: 1.0

```

Q9. Email Spam Detection

```

import numpy as np
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

np.random.seed(7)

texts=['Free money now','Meeting schedule','Win a prize','Project
update','Limited offer','Invoice attached','Claim your prize','Team
lunch','Cheap meds online','Report draft']

labels=[1,0,1,0,1,0,1,0,1,0]

df=pd.DataFrame({'text':texts,'spam':labels})

```

```

vec=TfidfVectorizer()
X=vec.fit_transform(df['text'])
y=df['spam']
model=MultinomialNB()
model.fit(X,y)
new=['Exclusive offer just for you','Please review the attached
report']
X_new=vec.transform(new)
print(model.predict(X_new))

```

```
[1 0]
```

Q10. Student Risk Prediction

```

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,accuracy_score
import matplotlib.pyplot as plt
np.random.seed(10)
n=300
attendance=np.random.randint(40,101,n)
mid_marks=np.random.randint(0,101,n)
final_marks=(mid_marks*0.5 + attendance*0.2 +
np.random.normal(0,10,n)).clip(0,100)
at_risk=(final_marks<40).astype(int)
df=pd.DataFrame({'attendance':attendance,'mid_marks':mid_marks,'final_marks':final_marks,'at_risk':at_risk})
X=df[['attendance','mid_marks','final_marks']]
y=df['at_risk']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
model=RandomForestClassifier(random_state=0)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

```

```

print('Accuracy:',accuracy_score(y_test,y_pred))

print(classification_report(y_test,y_pred))

importances=pd.Series(model.feature_importances_,index=X.columns).so
rt_values(ascending=False)

print(importances)

plt.figure(figsize=(6,4))

importances.plot(kind='bar')

plt.title('Feature Importance')

plt.ylabel('Importance')

plt.show()

```

```

Accuracy: 1.0
              precision    recall  f1-score   support

     0           1.00       1.00       1.00        24
     1           1.00       1.00       1.00        36

 accuracy               1.00               1.00        60
 macro avg              1.00               1.00        60
weighted avg              1.00               1.00        60

final_marks    0.598208
mid_marks      0.347948
attendance      0.053844
dtype: float64

```

